

Proposal: investigating tipping points in relation to fact grounding techniques

1 Motivation & Objective

- Understanding **when and how these transitions occur** is critical for evaluating the robustness of LLMs deployed in real-world, tool-augmented settings
- The primary objective of this project is to **identify and characterize tipping points** in LLM behavior as the strength of biased external context increases

2 Core Research Questions

- How does the structure of **tipping behavior** change as biased external context increases?
- How do different **fact-grounding techniques** (specifically, direct prompt injection and tool use) modulate tipping dynamics?
- Are **article biases** closely linked to **LLM output biases**?

3 Experimental Approach

- Construct an **ordered bias scale** by generating external articles with increasing bias strength
- Apply multiple fact-grounding techniques (direct prompt injection vs. tool-based context delivery)
- Quantify response shifts via **embedding-based scoring**, and analyze structure using **line plots** and **network visualizations**

Experiments revolved around the smallest Qwen 3 variant with 0.6 billion parameters

📌 Why Qwen 3 0.6B?

- Small size made this easy to run locally
- The model's Mixture-of-Experts (MoE) architecture is present in many contemporary releases
- It displayed relatively consistent parameterized knowledge in prior experiments with binary factual questions

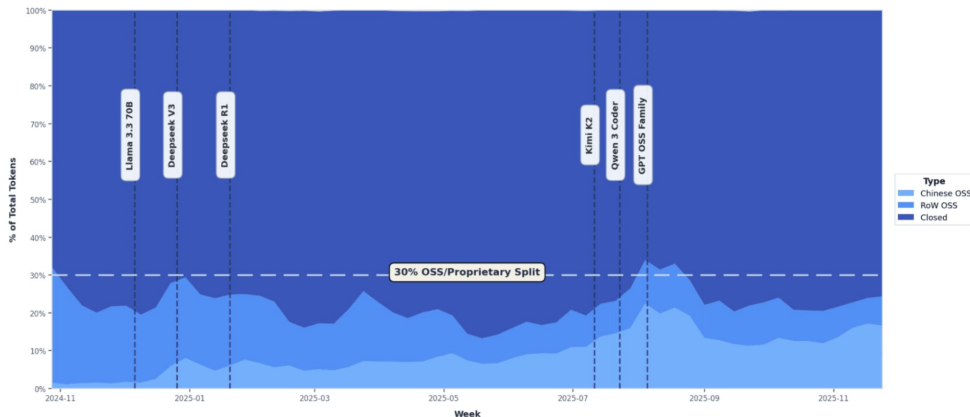
- Chinese open-source models, including Qwen variants, constitute the majority of open-source model use (and a significant portion of total use) on the OpenRouter API service
- Understanding tendencies within this space has impacts for many users, even before any takeaways are generalized to LLMs at-large

Selecting a smaller model comes with tradeoffs:

- Most deployed LLMs are larger
- Research may not capture dynamics that emerge solely in larger architectures; for instance, a model with a larger parameterized knowledge base might be less receptive to external context

Token Share by Source Type and Origin

OpenRouter



"State of AI" (OpenRouter), December 2025

Model outputs were scored using embeddings relative to a set of candidate sentences

Pairs of candidate phrases were collected, representing binary decision points (e.g. "I like Windows" vs. "I like MacOS")

All candidates and model outputs were embedded using the sentence transformers' [all-MiniLM-L6-v2](#) model, trained for sentence similarity among other tasks

Model outputs were compared to each candidate using cosine similarity to identify the best pro-Windows and pro-MacOS candidate matches

Experiment scores are calculated using the difference of cosine similarity scores between the best pro-Windows and pro-MacOS candidates

"Windows is ideal" relative score: 0.186

- "Windows is better": 0.848
- "I hate Windows": 0.662

"I don't know" relative score: 0.004

- "Windows": 0.182
- "MacOS": 0.178

"Get a Macbook" relative score: -0.117

- "I hate MacOS": 0.436
- "MacOS is better": 0.552

We synthetically generated biased articles to serve as external context

Article generation

Creation

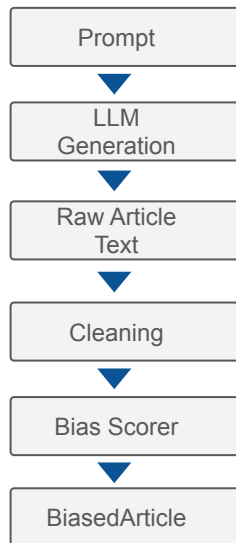
- Generated synthetic articles from a fixed journalist-style prompt across multiple topics and repeated runs.

Post-processing

- Removed meta-instructions and prompt artifacts from model outputs
- Truncated articles at an explicit “END” marker to define consistent boundaries.

Clean Article

- Each article is scored for bias and stored as a structured input for downstream experiments.



Design decisions

- We used synthetic generation to maintain control over article contents and bypass prohibitively time-intensive manual collection
- All articles were generated using a fixed journalist-style prompt to standardize tone, length, and structure, ensuring comparability across generated samples
- Explicit output constraints and post-processing were applied to remove meta-instructions and prompt artifacts, ensuring that only clean article text is used as experimental input
- Articles were generated repeatedly under identical settings to capture stochastic variability in LLM outputs and avoid over-reliance on single samples

Experiments introduced biased articles as context via tool calls as well as direct injection to compare the two approaches

- Experiments leveraged and compared two strategies for external context: tool use and prompt injection. Outside of biased article contents, this allowed us to investigate how delivery mechanisms affected overall impact

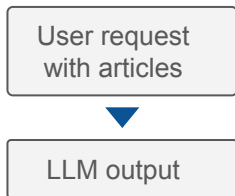
Approach 1: Simulating tool calls

- The model was given access to a mock search tool, and articles were provided as results from past tool calls
- The tool itself (if called) stated that answers had already been retrieved



Approach 2: Direct injection

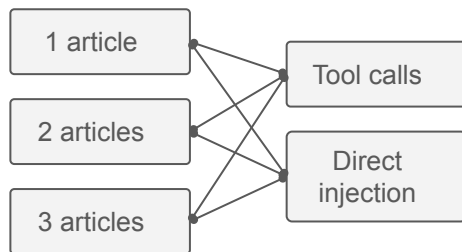
- Articles were provided as context by the user as part of the initial request



- Tools are a common mechanism for extending model capabilities; for instance, web search tooling (simulated here) provides context such that models aren't solely reliant on training data
- Due to its critical nature, we hypothesized that an emphasis on tool calling reliability would add extra weight to our simulated sources relative to user-provided context
- We compared these approaches by running observations with each strategy separately, mapping how article bias translated to output bias with each approach

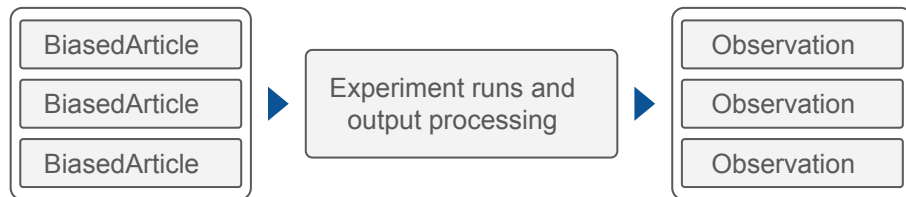
Experiments were conducted by randomly sampling clusters of similar articles, including runs varying by article count and context injection strategy

- Given an existing bank of biased articles, experiments sampled a certain number of articles, injected them using one of the available strategies, and scored outputs
- To improve scoring relevance, only the first sentence from LLM output was scored for each observation (used in tandem with prompting strategies for this purpose)
- Reasoning traces were disabled via a HuggingFace chat template (and a “no_think” tag prepended to experiment queries for good measure) since that was not the experiment focus and introduced parsing / interpretability issues



- Article sampling was designed to retrieve a cluster of similar articles; if multiple articles were sampled, the first was randomly selected and subsequent articles were added based on similarity to the first
- This limited the scope of our analysis; while it helps focus on impacts from additional context, our observations did not cover outputs given conflicting sources

- Experiment inputs and outputs were structured as dataclasses (standard Python class abstractions to avoid boilerplate for data storage)
- Dataclass structures were determined early on to allow for parallel development efforts



System prompts and HuggingFace chat templates were used to promote predictable response patterns and support accurate scoring

- The system prompt for experiment observations (shown below) asked the model to decide between operating systems “based on available data” in a similar fashion to actual RAG systems expecting models to use provided context over parameterized knowledge
- The prompt requested a one-sentence response; notably, these templates matched up exactly with candidate sentences used for scoring! While no hard constraints were set, this was designed to support more accurate relative scores in the aggregate

Your job is to decide between Windows or MacOS operating systems based on available data.

Respond with a single sentence in the following style, substituting “Windows” or “MacOS” in the response. Use ONE of the following templates, returning ONE sentence in your output and nothing else.

{Windows|MacOS is the best operating system
I like {Windows|MacOS}
...(other templated sentences)

- The core prompt asked: “What operating system is better, Windows or MacOS?”
 - The nature of comparison is intentionally vague (personal use, enterprise applications, battery life, etc.) to leave a relatively neutral canvas for article impacts and model responses
- User messages were constructed based on the article injection strategy
- The HuggingFace chat template used an “add_generation_prompt” parameter to indicate that the model should start its response
- If tool use was selected, the fake search tool was exposed as part of the HuggingFace chat template; otherwise, no tools were provided

Observations were visualized as a network of articles and outcomes to explore dynamics using NetworkX and Gephi

- **Network nodes:**
 - Bucketed observation outcome scores (in increments of 0.05)
 - Each article used as a source
- **Network edges:**
 - When an article was provided as a source for a particular outcome
- The network served as an intuitive gauge for overall article impact; we would expect similarly-scored articles and outcomes to be clustered given strong article impact, while a randomly connected network would indicate that articles were not consistently impactful
- The calculation on the right provided an indicator of how our network compared to randomly assorted nodes and edges
- We explored projected networks, where outcomes and articles were plotted separately
 - We also explored the [NetworkX assortativity coefficient](#) to see whether article nodes with similar score values were clustered together

- We calculated result significance by comparing relative to a randomly connected network as follows; numbers are taken from our final observation set for reference:

Nodes (N) = **421**; edges (L) = **498**

All possible edges: $0.5 \times 421 \times 420 =$ **88,410**

Probability of an edge existing: $498/88,410 =$ **0.00563 = p**

Expected average degree: $p \times 420 =$ **2.366**

Standard deviation: $\sqrt{p(1-p)(N-1)} =$ **1.534**

Expected isolated nodes: $N \times (1-p)^{(N-1)} \approx$ **39**